

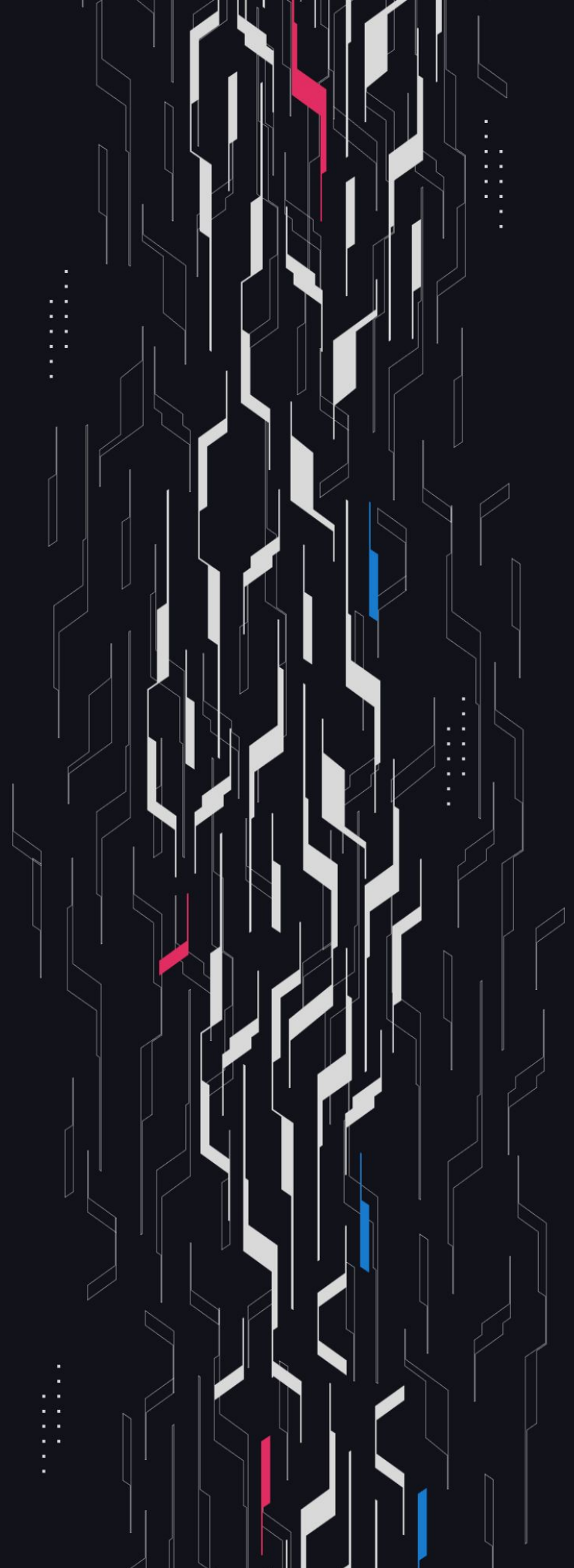
**GA GUARDIAN**

# Degensafe

**Protocol Review**

**Security Assessment**

**February 2nd, 2026**



# Summary

**Audit Firm** Guardian

**Prepared By** Daniel Gelfand, Owen Thurm

**Client Firm** Degensafe

**Final Report Date** February 2, 2026

## Audit Summary

Degensafe engaged Guardian to review the security of their Degensafe. From the 8th of December 2025 to the 11th of December 2025, a team of 2 auditors reviewed the source code in scope. All findings have been recorded in the following report.

## Confidence Ranking

Given the significant number of High and Critical issues detected as well as overall systemic flaws, Guardian assigns a Confidence Ranking of 1 to the protocol. Guardian strongly recommends that an independent security review of the protocol at a finalized frozen commit is conducted before deployment. Guardian strongly advises that the protocol undergo a full follow-up audit at a finalized and fully remediated commit before any mainnet deployment. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

**Security Recommendation:** Guardian recommends a follow-up security review of the protocol at a finalized frozen commit.

✓ Verify the authenticity of this report on Guardian's GitHub: <https://github.com/guardianaudits>

# Guardian Confidence Ranking

| Confidence Ranking      | Definition and Recommendation                                                                                                                                                                                                                                                                                                                                               | Risk Profile                                                                                                                                                                 |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5: Very High Confidence | <p>Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design.</p> <p><b>Recommendation:</b> Code is highly secure at time of audit. Low risk of latent critical issues.</p>                                                                                        | 0 High/Critical findings and few Low/Medium severity findings.                                                                                                               |
| 4: High Confidence      | <p>Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong.</p> <p><b>Recommendation:</b> Suitable for deployment after remediations; consider periodic review with changes.</p>                                                                             | 0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.                                                        |
| 3: Moderate Confidence  | <p>Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases.</p> <p><b>Recommendation:</b> Address issues thoroughly and consider a targeted follow-up audit depending on code changes.</p> | 1-2 High/Critical findings per engagement week.                                                                                                                              |
| 2: Low Confidence       | <p>Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues.</p> <p><b>Recommendation:</b> Post-audit development and a second audit cycle are strongly advised.</p>                                                  | 2-4 High/Critical findings per engagement week. Or additional High/Critical findings uncovered in remediation review which have not been resolved and confirmed by Guardian. |
| 1: Very Low Confidence  | <p>Code has systemic issues. Multiple High/Critical findings (<math>\geq 5</math>/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured.</p> <p><b>Recommendation:</b> Halt deployment and seek a comprehensive re-audit after substantial refactoring.</p>                                                             | $\geq 5$ High/Critical findings and overall systemic flaws.                                                                                                                  |

# Table of Contents

## Project Information

Project Overview ..... 5

Audit Scope & Methodology ..... 6

## Smart Contract Risk Assessment

Findings & Resolutions ..... 9

## Addendum

Disclaimer ..... 39

About Guardian ..... 40

# Project Overview

## Project Summary

|              |                                                                                                           |
|--------------|-----------------------------------------------------------------------------------------------------------|
| Project Name | Degensafe                                                                                                 |
| Language     | Solidity                                                                                                  |
| Codebase     | <a href="https://github.com/VasilVR/degensafe-programs">https://github.com/VasilVR/degensafe-programs</a> |
| Commit(s)    | Main Review commit: 4f34c5f333ccea882527d8a68fdee0600689a377                                              |

## Audit Summary

|                   |                                                              |
|-------------------|--------------------------------------------------------------|
| Delivery Date     | February 1, 2026                                             |
| Audit Methodology | Static Analysis, Manual Review, Test Suite, Contract Fuzzing |

## Vulnerability Summary

| Vulnerability Level | Total | Pending | Declined | Acknowledged | Partially Resolved | Resolved |
|---------------------|-------|---------|----------|--------------|--------------------|----------|
| ● Critical          | 5     | 0       | 0        | 0            | 0                  | 5        |
| ● High              | 2     | 1       | 0        | 0            | 0                  | 1        |
| ● Medium            | 4     | 1       | 0        | 0            | 0                  | 3        |
| ● Low               | 9     | 3       | 0        | 0            | 2                  | 4        |
| ● Info              | 7     | 1       | 0        | 0            | 2                  | 4        |

# Audit Scope & Methodology

**Contracts in Scope:**

staking.rs: 502 code | 31 comment | 112 blank  
lc\_deposit.rs: 348 code | 14 comment | 68 blank  
sol\_deposit.rs: 196 code | 13 comment | 41 blank  
  
TOTAL: 1046 code | 58 comment | 221 blank

# Audit Scope & Methodology

## Vulnerability Classifications

| Severity                  | Impact: <i>High</i> | Impact: <i>Medium</i> | Impact: <i>Low</i> |
|---------------------------|---------------------|-----------------------|--------------------|
| Likelihood: <i>High</i>   | ● Critical          | ● High                | ● Medium           |
| Likelihood: <i>Medium</i> | ● High              | ● Medium              | ● Low              |
| Likelihood: <i>Low</i>    | ● Medium            | ● Low                 | ● Low              |

## Impact

- High** Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.
- Medium** A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.
- Low** Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

## Likelihood

- High** The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.
- Medium** An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.
- Low** Unlikely to ever occur in production.

# Audit Scope & Methodology

## **Methodology**

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.  
Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.



# Findings & Resolutions

| ID                   | Title                                           | Category      | Severity   | Status     |
|----------------------|-------------------------------------------------|---------------|------------|------------|
| <a href="#">C-01</a> | Anyone Can Takeover Pool And Corrupt State      | Logical Error | ● Critical | Resolved   |
| <a href="#">C-02</a> | Attacker Can Drain Arbitrary User Stake         | Logical Error | ● Critical | Resolved   |
| <a href="#">C-03</a> | Vault PDA Collision Enables Cross-Pool Theft    | Logical Error | ● Critical | Resolved   |
| <a href="#">C-04</a> | Vault Authority Takeover                        | Logical Error | ● Critical | Resolved   |
| <a href="#">C-05</a> | Authority Hijacking In SOL Deposit              | Logical Error | ● Critical | Resolved   |
| <a href="#">H-01</a> | Varied Reward And Deposit Token Leads To Drain  | Logical Error | ● High     | Unresolved |
| <a href="#">H-02</a> | Deposit Record Grief Through Order ID Creation  | Griefing      | ● High     | Resolved   |
| <a href="#">M-01</a> | Reward Rate Changes Apply Retroactively         | Logical Error | ● Medium   | Resolved   |
| <a href="#">M-02</a> | Withdrawal Destination Not Validated            | Logical Error | ● Medium   | Resolved   |
| <a href="#">M-03</a> | Initialize Frontrunning Risk                    | Frontrunning  | ● Medium   | Unresolved |
| <a href="#">M-04</a> | Insufficient Rewards Block Principal Withdrawal | DoS           | ● Medium   | Resolved   |
| <a href="#">L-01</a> | Fractional Reward Rates Not Supported           | Logical Error | ● Low      | Resolved   |
| <a href="#">L-02</a> | Withdrawal Wallet Argument And Account Mismatch | Validation    | ● Low      | Resolved   |

# Findings & Resolutions

| ID                   | Title                                            | Category       | Severity | Status             |
|----------------------|--------------------------------------------------|----------------|----------|--------------------|
| <a href="#">L-03</a> | Token Mint Not Validated In SetWithdrawalAccount | Validation     | ● Low    | Resolved           |
| <a href="#">L-04</a> | Loose ATA Existence Check                        | Validation     | ● Low    | Partially Resolved |
| <a href="#">L-05</a> | Balance Tracking Desync With Direct Transfer     | Logical Error  | ● Low    | Resolved           |
| <a href="#">L-06</a> | ATA Address Not Validated                        | Validation     | ● Low    | Partially Resolved |
| <a href="#">L-07</a> | Reward Capping For High-Decimal Tokens           | Logical Error  | ● Low    | Unresolved         |
| <a href="#">L-08</a> | Order ID Length Not Validated                    | Validation     | ● Low    | Unresolved         |
| <a href="#">L-09</a> | Backend Must Act Accordingly                     | Warning        | ● Low    | Unresolved         |
| <a href="#">I-01</a> | Unbounded Reward Percentage                      | Validation     | ● Info   | Resolved           |
| <a href="#">I-02</a> | Unused DepositNotFound Error                     | Best Practices | ● Info   | Resolved           |
| <a href="#">I-03</a> | First Deposit Requires Rent-Exempt Amount        | Warning        | ● Info   | Unresolved         |
| <a href="#">I-04</a> | Missing PDA Seed Validation On Pool Account      | Warning        | ● Info   | Partially Resolved |
| <a href="#">I-05</a> | Reward Claiming Requires Zero Amount Withdrawal  | Logical Error  | ● Info   | Resolved           |
| <a href="#">I-06</a> | Single Pool Per Token Mint Limitation            | Documentation  | ● Info   | Resolved           |

# Findings & Resolutions

| ID                   | Title                            | Category | Severity | Status             |
|----------------------|----------------------------------|----------|----------|--------------------|
| <a href="#">I-07</a> | Inconsistent ATA Existence Check | Warning  | ● Info   | Partially Resolved |

# C-01 | Anyone Can Takeover Pool And Corrupt State

| Category      | Severity   | Location       | Status   |
|---------------|------------|----------------|----------|
| Logical Error | ● Critical | staking.rs: 10 | Resolved |

## Description

The `create_pool` instruction uses `init_if_needed` for the pool account, which allows the instruction to succeed even when the pool already exists. While `init_if_needed` prevents re-allocation of an existing account, the function unconditionally overwrites all pool state:

```
pool.owner = maybe_owner.unwrap_or(ctx.accounts.admin.key());
pool.token_mint = ctx.accounts.token_mint.key();
pool.reward_mint = ctx.accounts.reward_mint.key();
pool.reward_percentage = reward_percentage;
pool.total_staked = 0;
pool.bump = ctx.bumps.pool;
pool.reward_vault = ctx.accounts.reward_vault.key();
pool.is_active = true;
```

There is no check verifying that the pool is uninitialized or that the caller is the existing owner. Any user can call `create_pool` on an existing pool and:

- Seize ownership to set themselves as `pool.owner`
- Reset `total_staked` to 0 -> causes underflow panics on subsequent withdrawals when `pool.total_staked.checked_sub(amount)` is called
- Change `reward_mint` and `reward_vault` -> redirect rewards or cause withdrawal failures
- Modify `reward_percentage` -> manipulate reward calculations

## Recommendation

Replace `init_if_needed` with `init` to ensure the instruction fails if the pool already exists.

## Resolution

Degensafe Team: Resolved.

# C-02 | Attacker Can Drain Arbitrary User Stake

| Category      | Severity   | Location        | Status   |
|---------------|------------|-----------------|----------|
| Logical Error | ● Critical | staking.rs: 245 | Resolved |

## Description

The `withdraw_stake` instruction only checks that `user_stake.amount >= amount` and lacks any further validation on the `user_stake` account. An attacker can exploit this by passing any victim's `UserStake` account while providing their own token accounts as the withdrawal destination.

Ultimately, an arbitrary attacker can steal other user's tokens/rewards to their own accounts, and drain arbitrary pools.

## Recommendation

Add constraints that `user_stake.owner == user.key()` and `user_stake.pool == pool.key()`.

## Resolution

Degensafe Team: Resolved.

# C-03 | Vault PDA Collision Enables Cross-Pool Theft

| Category      | Severity   | Location        | Status   |
|---------------|------------|-----------------|----------|
| Logical Error | ● Critical | staking.rs: 335 | Resolved |

## Description

The `reward_vault` PDA is derived only from the reward mint, not the pool. When multiple pools use the same reward token, they all share a single vault, but only the first pool to create it becomes the token account authority.

Consider the following scenario:

- Admin A creates Pool A with `token_mint = SOL` and `reward_mint = USDC`
- The `reward_vault` PDA is created with Pool A as authority
- Admin B creates Pool B with `token_mint = ETH` and `reward_mint = USDC`
- The vault already exists; `init_if_needed` skips initialization
- Pool A remains the authority
- Admin B deposits 100,000 USDC rewards into the shared vault
- Admin A calls `withdraw_reward` for 100,000 USDC
- The program signs with Pool A's seeds, which matches the vault authority
- Admin A receives all of Pool B's rewards

Ultimately Pool B's users stake tokens expecting rewards, but the vault has been drained by an unrelated pool owner.

## Recommendation

Include the pool address in the reward vault PDA seeds to ensure each pool has its own isolated vault

## Resolution

Degensafe Team: Resolved.

# C-04 | Vault Authority Takeover

| Category      | Severity   | Location          | Status   |
|---------------|------------|-------------------|----------|
| Logical Error | ● Critical | lc_deposit.rs: 14 | Resolved |

## Description

The `initialize` instruction uses `init_if_needed` for the `vault_state` account without verifying whether the vault has already been initialized. While `init_if_needed` prevents re-allocation of an existing account, it does not prevent `initialize` from being called again and overwriting fields on an existing account.

Any user can call `initialize` on an existing vault and:

- 1. Seize ownership -> overwrite `authority` with their own pubkey
- 2. Set `wallet_account` to themselves -> drain all tokens to themselves through `withdraw`
- 3. Corrupt accounting -> reset `balance` to 0

Once an attacker becomes the authority, they gain full control over `set_withdrawal_account` and `withdraw`, allowing them to drain all deposited funds.

## Recommendation

Replace `vault_state`'s `init_if_needed` with `init` to ensure the instruction fails if the vault already exists or validate against an `is_initialized` boolean which is set to `true` on the first `initialize` call.

## Resolution

Degensafe Team: Resolved.

# C-05 | Authority Hijacking In SOL Deposit

| Category      | Severity   | Location           | Status   |
|---------------|------------|--------------------|----------|
| Logical Error | ● Critical | sol_deposit.rs: 11 | Resolved |

## Description

The `initialize` function uses `init_if_needed` to create the vault state account, but unconditionally overwrites all state fields including the `authority` on every call.

Since the vault state is a PDA derived from a static seed (`b"vault_state"`), the account will already exist after the first legitimate initialization.

However, because `init_if_needed` allows the instruction to succeed even when the account exists, any attacker can call `initialize()` to overwrite the `authority` field with their own public key.

This grants the attacker full administrative control over the vault, allowing them to set the withdrawal wallet and drain all deposited funds.

## Recommendation

Replace `init_if_needed` with `init` to ensure the instruction can only succeed once.

## Resolution

Degensafe Team: Resolved.



# H-01 | Varied Reward And Deposit Token Leads To Drain

| Category      | Severity | Location        | Status     |
|---------------|----------|-----------------|------------|
| Logical Error | ● High   | staking.rs: 606 | Unresolved |

## Description

The reward calculation treats staked token base units as equivalent to reward token base units, with no price or value conversion. This fundamental flaw causes mispayments when the reward token has a different value than the staking token.

The formula simply computes:  $\text{reward} = \text{staked\_amount} * \text{percentage} * \text{time} / \text{year}$   
This assumes 1 base unit staked = 1 base unit rewarded, ignoring that tokens have entirely different valuations.

Underpayment Scenario:

- Staking token: WBTC (8 decimals, ~\$100,000 per coin)
  - Reward token: USDC (6 decimals, \$1 per coin)
  - Pool configured with 10% APY
  - User stakes 1 WBTC = 100,000,000 base units (\$100,000 value)
  - After 1 year, reward calculation:  $100,000,000 * 10 / 100 = 10,000,000$  base units
  - Result: User receives 10,000,000 USDC base units = 10 USDC
- Expected: \$10,000 in rewards (10% of \$100,000)  
Actual: \$10 in rewards

Overpayment Scenario:

- Staking token: USDC (6 decimals)
  - Reward token: WBTC (8 decimals)
  - User stakes 1,000 USDC = 1,000,000,000 base units (\$1,000 value)
  - After 1 year:  $1,000,000,000 * 10 / 100 = 100,000,000$  base units
  - Result: User receives 100,000,000 WBTC base units = 1 WBTC = \$100,000
- Expected: \$100 in rewards (10% of \$1,000)  
Actual: \$100,000 in rewards — a 1000x overpayment

A single user staking \$1,000 USDC for one year drains \$100,000 from the reward vault.

## Recommendation

Implement a price oracle or admin-configured exchange rate to convert between staking token value and reward token value. Alternatively, restrict the protocol to same-token staking and rewards

## Resolution

Degensafe Team: Unresolved.

# H-02 | Deposit Record Grief Through Order ID Creation

| Category | Severity | Location | Status   |
|----------|----------|----------|----------|
| Griefing | ● High   | Global   | Resolved |

## Description

The DepositRecord PDA seeds in both `lc_deposit` and `sol_deposit` do not include the depositor's address, allowing any user to claim arbitrary `order_id` values and block legitimate deposits.

In `lc_deposit`, the seeds are: `seeds = [b"deposit_record", vault_state.token_mint.as_ref(), order_id.as_bytes()]`.

In `sol_deposit`, the seeds are: `seeds = [b"deposit_record", order_id.as_bytes()]`.

Since `user` is not part of the PDA derivation, an attacker can front-run or preemptively create deposit records for any `order_id` with minimal cost. The `init` constraint causes the victim's transaction to fail when the PDA already exists, blocking specific users from depositing.

This is especially impactful since token creation fees must be paid in both `sol_deposit` and `lc_deposit` each time, and the `order_id`'s must match between the two programs.

Therefore, after a user deposits in one vault, an attacker can take the `order_id` in the other vault for minimal cost – preventing the token creation process from fully going through and locking user funds.

## Recommendation

Consider including the user's pubkey in the PDA seeds.

## Resolution

Degensafe Team: Resolved.

# M-01 | Reward Rate Changes Apply Retroactively

| Category      | Severity | Location       | Status   |
|---------------|----------|----------------|----------|
| Logical Error | ● Medium | staking.rs: 88 | Resolved |

## Description

The reward calculation uses the current `reward_percentage` multiplied by the entire elapsed time since `last_staked_time`. When an admin updates the reward rate, it retroactively applies to all previously accrued but unclaimed rewards.

Consider the following scenario:

- User stakes 10,000 tokens at 20% APY
- User waits 364 days, expecting ~2,000 tokens in rewards
- Admin changes rate to 1% APY on day 364
- User withdraws on day 365
- Calculation uses 1% for the entire 365 days
- User receives ~100 tokens instead of ~2,000

## Recommendation

Snapshot pending rewards before applying rate changes.

## Resolution

Degensafe Team: Resolved.

# M-02 | Withdrawal Destination Not Validated

| Category      | Severity | Location           | Status   |
|---------------|----------|--------------------|----------|
| Logical Error | ● Medium | sol_deposit.rs: 61 | Resolved |

## Description

The `withdraw` function checks that a withdrawal wallet has been configured in the vault state, but the `wallet_account` passed into the instruction is never validated against this stored value. The account is marked with `/// CHECK` and only constrained to be mutable.

The authority can pass any arbitrary address as `wallet_account`, and the funds will be transferred to that address regardless of what was configured with `set_withdrawal_account`, rendering the withdrawal wallet configuration meaningless.

## Recommendation

Add `has_one = wallet_account` to the `vault_state` account constraints in the `Withdraw` struct.

## Resolution

Degensafe Team: Resolved.

# M-03 | Initialize Frontrunning Risk

| Category     | Severity | Location | Status     |
|--------------|----------|----------|------------|
| Frontrunning | ● Medium | Global   | Unresolved |

## Description

Deployment of the program and the call to initialize on Solana are not atomic -- meaning the program is first deployed, and initialize must be called in a separate transaction.

Since initialize lacks any access control, anyone can call it immediately after deployment. A malicious actor could front-run the intended initializer, set the authority to their own address, and take full control of the protocol's configuration and operations.

## Recommendation

Deploy and call initialize in the same transaction using a deployment script.

## Resolution

Degensafe Team: Unresolved.

# M-04 | Insufficient Rewards Block Principal Withdrawal

| Category | Severity | Location        | Status   |
|----------|----------|-----------------|----------|
| DoS      | ● Medium | staking.rs: 262 | Resolved |

## Description

In `withdraw_stake`, the function checks if the reward vault has sufficient balance to pay rewards before allowing any withdrawal:

```
require!(
  ctx.accounts.reward_vault.amount >= reward_to_send,
  CustomError::InsufficientRewardVault
);
```

If the reward vault is underfunded, users cannot withdraw their staked principal even though the principal tokens exist in a separate `pool_vault`. This creates a situation where user funds can be locked if the admin fails to maintain adequate reward token reserves or drains the reward vault.

## Recommendation

Consider allowing principal withdrawal even when rewards cannot be paid. Otherwise, clearly document this risk.

## Resolution

Degensafe Team: Resolved.

# L-01 | Fractional Reward Rates Not Supported

| Category      | Severity | Location        | Status   |
|---------------|----------|-----------------|----------|
| Logical Error | ● Low    | staking.rs: 630 | Resolved |

## Description

The `reward_percentage` field is stored as a `u64` and interpreted as a whole number percentage, dividing by 100 in the reward calculation. This prevents configuring fractional APY rates (e.g. 5.5%).

## Recommendation

Consider using basis points instead of percentages directly.

## Resolution

Degensafe Team: Resolved.

# L-02 | Withdrawal Wallet Argument And Account Mismatch

| Category   | Severity | Location          | Status   |
|------------|----------|-------------------|----------|
| Validation | ● Low    | lc_deposit.rs: 63 | Resolved |

## Description

The `set_withdrawal_account` function accepts both a `new_wallet: Pubkey` argument and a `ctx.accounts.new_wallet` account. The vault state stores the argument value, but the ATA creation uses the account as the authority.

If the caller passes mismatched values:

- `new_wallet` arg = address A
- `ctx.accounts.new_wallet` account = address B

The vault state will point to A, but the ATA is created for B. When `withdraw` is called, it expects an ATA with authority matching `vault_state.wallet_account` (A), which doesn't exist. Withdrawals fail until the admin calls `set_withdrawal_account` again with matching values.

## Recommendation

Enforce the account matches passed `new_wallet` argument.

## Resolution

Degensafe Team: Resolved.



# L-03 | Token Mint Not Validated In SetWithdrawalAccount

| Category   | Severity | Location          | Status   |
|------------|----------|-------------------|----------|
| Validation | ● Low    | lc_deposit.rs: 89 | Resolved |

## Description

The SetWithdrawalAccount context accepts a token\_mint account but does not constrain it to match vault\_state.token\_mint. If the wrong mint is passed, the function creates an ATA for an unrelated token which can trigger reverts when calling withdraw.

## Recommendation

Add a constraint to ensure the provided mint matches the vault's configured mint.

## Resolution

Degensafe Team: Resolved.

# L-04 | Loose ATA Existence Check

| Category   | Severity | Location           | Status             |
|------------|----------|--------------------|--------------------|
| Validation | ● Low    | lc_deposit.rs: 190 | Partially Resolved |

## Description

Both `set_withdrawal_account` and `create_wallet_ata_if_needed` compute the canonical ATA address but use weak checks to determine if the ATA already exists:

- `set_withdrawal_account` checks if the account is owned by the Token program
- `create_wallet_ata_if_needed` checks if the account is not owned by the System program

Neither verifies that the passed `associated_token` account actually matches the derived canonical ATA address. Any account passing these ownership checks is treated as the valid ATA, including unrelated token accounts, other wallets' ATAs, or accounts for different mints.

## Recommendation

Verify the passed account address matches the canonical ATA before checking existence. Explicitly handle the three cases: Token program owned (exists), System program owned (needs creation), and other (reject).

## Resolution

Degensafe Team: Partially Resolved.

# L-05 | Balance Tracking Desync With Direct Transfer

| Category      | Severity | Location           | Status   |
|---------------|----------|--------------------|----------|
| Logical Error | ● Low    | sol_deposit.rs: 76 | Resolved |

## Description

The sol\_deposit and lc\_deposit programs maintain a balance field in vault\_state that is incremented during deposits and set to zero during withdrawals. However, the actual withdrawal transfers the full lamport balance of vault\_pda, not the tracked balance value.

If SOL/SPL token is sent directly to vault\_pda outside of the deposit instruction, the vault\_state.balance will not reflect the true holdings. This creates a desync between tracked and actual balances and may confuse integrators/DegenSafe’s backend component.

## Recommendation

Consider removing the redundant balance tracking and rely solely on the PDA's lamport balance and token amount in the vault ATA.

## Resolution

Degensafe Team: Resolved.

# L-06 | ATA Address Not Validated

| Category   | Severity | Location          | Status     |
|------------|----------|-------------------|------------|
| Validation | ● Low    | lc_deposit.rs: 70 | Unresolved |

## Description

Both `set_withdrawal_account` and `create_wallet_ata_if_needed` compute the canonical ATA address using `get_associated_token_address`, but neither verifies that the passed `associated_token` account actually matches this derived address.

A caller could pass any arbitrary account as `associated_token`. The function would either skip creation (if the account passes the loose existence check) or attempt to create an ATA at the wrong address.

## Recommendation

Add a check to verify the passed associated token account matches the calculated ATA.

## Resolution

Degensafe Team: Unresolved.

# L-07 | Reward Capping For High-Decimal Tokens

| Category      | Severity | Location   | Status     |
|---------------|----------|------------|------------|
| Logical Error | ● Low    | staking.rs | Unresolved |

## Description

The reward calculation uses u128 for intermediate arithmetic but clamps to u64 before transfer: `reward.min(u64::MAX as u128) as u64`.

For tokens with high decimals (e.g., 18 decimals), u64 can only represent ~18.44 whole tokens. If such tokens are used, rewards exceeding `u64::MAX` would be capped rather than failing, resulting in users receiving less than earned.

## Recommendation

Document that this system is only intended for usage with lower decimal tokens.

## Resolution

Degensafe Team: Unresolved.

# L-08 | Order ID Length Not Validated

| Category   | Severity | Location | Status     |
|------------|----------|----------|------------|
| Validation | ● Low    | Global   | Unresolved |

## Description

The deposit function in both `lc_deposit` and `sol_deposit` programs accepts an `order_id` parameter of type `String` without validating its length. The `DepositRecord` account allocates a fixed 64 bytes for the order ID string content, but no runtime check enforces this limit.

When a user submits an order ID exceeding 64 bytes, the transaction fails during account serialization because the data cannot fit in the allocated space. This results in a confusing error message rather than a clear validation failure

## Recommendation

Consider adding explicit length validation at the start of the `deposit` function in both programs.

## Resolution

Degensafe Team: Unresolved.

# L-09 | Backend Must Act Accordingly

| Category | Severity | Location | Status     |
|----------|----------|----------|------------|
| Warning  | ● Low    | Global   | Unresolved |

## Description

The backend systems consuming the on-chain data exposed through functions such as `check_deposit`, `get_user_stake_info`, etc. must validate and interpret it correctly. Consider some of the potential risks below that depend on backend implementation:

1. Deposit Record Reuse
  - Deposit records (`DepositRecord`) persist on-chain indefinitely after creation
  - If the backend does not mark deposits as "consumed" after token creation, an attacker could potentially reuse the same deposit proof multiple times
  - Impact: Double-spending of deposit credits, unlimited token creation from single payment
2. Depositor Identity Verification
  - The `user` field in deposit records shows who made the deposit, but contracts don't enforce that the requester matches the depositor
  - If backend doesn't verify `deposit_record.user == requesting_user`, one user could claim another's deposit
  - Impact: Theft of payment credits, unauthorized token creation
3. Timestamp/Expiry Validation
  - Deposit records include `timestamp` but contracts don't enforce expiry
  - If backend doesn't check deposit freshness, old deposits could be used indefinitely
  - Impact: Stale payment exploitation, accounting issues
4. Flash-Staking Exploitation
  - `last_staked_time` is provided but no minimum lock period is enforced on-chain
  - If off-chain systems grant benefits (airdrops, governance, tier access) based solely on `amount > 0` without checking staking duration, attackers can:
    - Stake immediately before snapshots
    - Qualify for benefits
    - Withdraw immediately after
  - Impact: Unfair distribution of rewards, governance manipulation
5. Balance Desynchronization
  - `vault_state.balance` tracks deposits through the program, but actual on-chain balance may differ if funds are sent directly
  - If backend relies on `vault_state.balance` instead of actual account balance, accounting errors occur
  - Impact: Incorrect financial reporting, potential loss tracking

## Recommendation

Ensure the backend has been appropriately reviewed as it plays a key role in the processing of on-chain data.

## Resolution

Degensafe Team: Unresolved.

# I-01 | Unbounded Reward Percentage

| Category   | Severity | Location        | Status   |
|------------|----------|-----------------|----------|
| Validation | ● Info   | staking.rs: 100 | Resolved |

## Description

The `update_reward_percentage` function accepts any `u64` value without validation. An admin can set `reward_percentage` to an arbitrarily high value, causing the reward calculation to produce extreme payouts.

## Recommendation

Consider enforcing validation for the reward percentage on-chain.

## Resolution

Degensafe Team: Resolved.



# I-02 | Unused DepositNotFound Error

| Category       | Severity | Location            | Status   |
|----------------|----------|---------------------|----------|
| Best Practices | ● Info   | sol_deposit.rs: 247 | Resolved |

## Description

The `VaultError` enum defines a `DepositNotFound` variant that is never used anywhere in the program logic.

## Recommendation

Remove the unused error variant to reduce code clutter, or implement it in `check_deposit` if the intent was to return a custom error for missing records.

## Resolution

Degensafe Team: Resolved.

# I-03 | First Deposit Requires Rent-Exempt Amount

| Category | Severity | Location           | Status     |
|----------|----------|--------------------|------------|
| Warning  | ● Info   | sol_deposit.rs: 30 | Unresolved |

## Description

The vault\_pda account is declared as an unchecked AccountInfo with no initialization logic. On the first deposit, this PDA does not exist on-chain and holds zero lamports.

The System Program's transfer instruction will fail if the destination account would fall below the rent-exempt minimum after the transfer, effectively requiring the first depositor to send at least the rent-exempt amount to bootstrap the PDA.

Hence the first deposit actually has a hidden minimum that the amount > 0 validation does not encompass, and that subsequent deposits do not have to follow.

## Recommendation

Explicitly initialize the vault\_pda and/or document this behavior clearly.

## Resolution

Degensafe Team: Unresolved.

# I-04 | Missing PDA Seed Validation On Pool Account

| Category | Severity | Location   | Status             |
|----------|----------|------------|--------------------|
| Warning  | ● Info   | staking.rs | Partially Resolved |

## Description

Multiple instruction account structs in the staking program lack PDA seed validation on the pool account.

While CreatePool correctly derives the pool PDA using seeds [b"staking\_pool", token\_mint.key().as\_ref()], subsequent instructions only use #[account(mut)] without verifying the pool address matches the expected PDA derivation.

Note that impact is limited since Pool is a typed Account<'info, Pool>, hence Anchor already enforces program ownership.

Structs missing seeds constraint on pool:

- GetPoolInfo - read-only, no owner check
- UpdateRewardMint - runtime owner check
- DepositReward - runtime owner check
- GetUserStakeInfo - read-only, no owner check
- WithdrawStake - no owner check
- WithdrawReward - runtime owner check
- UpdateRewardPercentage - runtime owner check
- SetStakingActive - runtime owner check
- DepositStake - no owner check

## Recommendation

Consider adding the seeds for the pool account in the various structs.

## Resolution

Degensafe Team: Partially Resolved.

# I-05 | Reward Claiming Requires Zero Amount Withdrawal

| Category      | Severity | Location        | Status   |
|---------------|----------|-----------------|----------|
| Logical Error | ● Info   | staking.rs: 245 | Resolved |

## Description

To claim accumulated rewards without unstaking principal, users must call `withdraw_stake(0)`. This may be unintuitive for users and should be clearly documented.

## Recommendation

Clearly document this behavior.

## Resolution

Degensafe Team: Resolved.

# I-06 | Single Pool Per Token Mint Limitation

| Category      | Severity | Location   | Status   |
|---------------|----------|------------|----------|
| Documentation | ● Info   | staking.rs | Resolved |

## Description

The pool PDA is derived using seeds `[b"staking_pool", token_mint.key().as_ref()]`, which means only one staking pool can exist per token mint. This is a design constraint that prevents creating multiple pools with different reward configurations for the same token.

## Recommendation

Clearly document this constraint.

## Resolution

Degensafe Team: Resolved.

# I-07 | Inconsistent ATA Existence Check

| Category | Severity | Location           | Status             |
|----------|----------|--------------------|--------------------|
| Warning  | ● Info   | lc_deposit.rs: 204 | Partially Resolved |

## Description

The codebase uses two different methods to determine whether an Associated Token Account (ATA) already exists.

In `set_withdrawal_account`, the check verifies ownership by the Token Program specifically, while `create_wallet_ata_if_needed` uses a looser check that assumes any account not owned by the System Program is an existing ATA.

The loose check would incorrectly skip ATA creation if the account were owned by any program other than System, not just the Token Program.

## Recommendation

Standardize on the stricter ownership check (verifying `owner == token::ID`) and consider validating the generated ata address against the passed account if exists.

## Resolution

Degensafe Team: Partially Resolved.

# Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

# About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit <https://guardianaudits.com>

To view our audit portfolio, visit <https://github.com/guardianaudits>

To book an audit, message <https://t.me/guardianaudits>